

THE WHOLE PROJECT, START TO FINISH

A NEXT GENT

A Linux computer made simple — and the business that grows out of making it simple.

THE BOX

Ghost

VERSION ONE

Ghost of Lisa

BUYER

Small business

MODEL

Red Hat

THESIS

Churn absorption

CONTENTS

01	What it is	12	How the money works
02	Where it starts	13	The box itself
03	What Red Hat actually sells	14	The harness
04	The wedge: absorbing churn	15	The name and the fight
05	The mechanism: a real phone	16	Who buys it, and how
06	Proof, not success	17	What's owned, what's given away
07	The way in: email	18	What exists today
08	What the data becomes	19	The order to build it
09	Four surfaces, one truth	20	The one test that decides everything
10	The apps on top	21	If it works
11	The platform and the store		

What it is

A small computer that sits in a business — on a wall, under a counter — and operates that business's software for them. The hours, the prices, the menu, the posts, the bookings. The owner speaks or texts in plain English and it goes and does the thing, on every platform at once, and then proves it did.

It runs Linux. The owner never finds that out. What they see is a screen laid out like a smart TV, because that is a grammar every adult already reads without being taught.

They buy the box once and subscribe to the updates. The subscription is not for the software — the software is open source and free. It is for the fact that when an app they depend on redesigns itself, somebody else fixes it before they notice.

That is the product. Everything else in this document is either how that works, or what becomes possible once it does.

Where it starts

In 2013 there was an email chain about building a directory to compete with Google. The pitch was simple and, at the time, unfashionable: nobody cares about 1.8 trillion search results. A person with a leaking faucet wants a plumber who can come today. They do not want to call four numbers and leave three voicemails.

The idea was correct and unbuildable. A directory is only as good as its data, and in 2013 getting live, truthful, item-level data out of thousands of small businesses was impossible. There was no mechanism. The businesses had nothing to publish with, and nobody would do data entry forever for free.

Fourteen years later the answer turns out not to be a better directory. It is to give the businesses something they want badly enough to run every day — something that saves them real money and real time — and to let the data fall out of it as a by-product.

THE REFRAME

The box is not the product. The box is the **mechanism that finally makes the data exist**. Everything that has ever been hard about local search becomes easy once thousands of businesses are publishing live truth about themselves because it is in their own interest.

What Red Hat actually sells

Most people get this wrong, and the entire business model depends on getting it right.

Red Hat does not sell software. Every line of it is free and public, and legally they cannot stop anyone from taking it. What a subscription buys is the ongoing work of keeping it correct: tested builds, security patches, a version that will not break next Tuesday, and someone on the hook when it does.

The customer is not buying code. They are buying **not having to do the integration work themselves**.

The enforcement is simple and it is not a lock. A paid account is what grants access to the update channel. Stop paying and nothing breaks — the machine runs forever. You keep everything you have. You stop receiving what comes next.

IBM paid \$34 billion for that arrangement. It is the largest software acquisition in history, and what they bought was a company that gives its product away.

What Red Hat sells	the build, not the code
What enforces it	entitlement, not DRM
What happens if you lapse	it keeps working
Upstream projects in RHEL	~2,500
Upstream projects here	~80

The wedge: absorbing churn

Red Hat absorbs kernel and package churn so an enterprise does not have to. That churn is scheduled, announced and slow. Enterprises could track it themselves; they pay not to.

A small business has a different upstream, and it is worse. Toast redesigns a screen on a Tuesday with no notice. Google moves a button. FareHarbor changes a confirmation email template. Every do-it-yourself automation in the country breaks silently, and nobody finds out until a customer shows up to a closed door.

That churn is faster, unannounced, and invisible until it has already cost something. It is a far better thing to sell protection from.

How the absorption actually runs

Every night, on every machine, every recorded procedure checks that the screens it expects are still where it left them. It touches nothing — it just looks. When one has moved, the machine refuses to act and reports it.

That report lands on one desk. One person re-records the path once. The fix is bundled, signed, and on four hundred machines by morning. **The marginal cost of fixing it for customer four hundred is zero, because it was already fixed for customer one.**

You are not selling software. You are
selling the fact that somebody absorbs
every app redesign, forever, so the
owner never thinks about it again.

THIS IS THE
SUBSCRIPTION

The mechanism: a real phone

The obvious way to update a business's hours everywhere is to integrate with each platform's API. That path is closed, and it is closed on purpose.

Toast's API is partner-gated. Google Business Profile needs quota approval. Most of the software a small business actually runs — the salon scheduler, the charter booking tool, the local POS — will never have an API anyone can get. Building integrations means asking permission from the companies with the most to lose.

So the integration is not an integration. It is a forty-dollar Android phone, bought by the owner, with the owner's SIM card and the owner's logins, sitting in a dock. It is remote-controlled at human speed. It taps the same buttons a person would.

Why this defeats every objection

- **Nothing to integrate.** Support for an app exists the moment the owner installs it. The long tail nobody would ever build a connector for works on day one.
- **Authentication solves itself.** They log in once, on a phone, the way they have a thousand times. A real SIM means SMS two-factor just works — the thing that breaks every other automation product.
- **It is a real device.** Emulators and containers get detected and blocked by exactly the banking and point-of-sale apps that matter. A physical handset with a real IMEI is indistinguishable from a person's phone, because it is one.
- **They own it.** Not shipped in the box, not leased. They buy it, they put their own SIM in it, and that is what makes it feel like theirs rather than yours.

THE DISTINCTION THAT MATTERS

It is not automation, and it is not an agent taking over. **It is remote control.** The owner's own account, on the owner's own device, with the owner's own credentials, on the owner's behalf, at human speed, with a person approving anything that matters.

Proof, not success

Automation that drives a user interface is brittle. Anyone who tells you otherwise has not run one for a year. Two rules turn brittle into shippable, and together they are the most defensible thing in the whole build.

One: refuse to act on a screen you do not recognise

Before the first tap, the machine checks that one or two specific elements are on screen where the recording says they should be. If they are not, it **stops and reports**. It does not guess, and it does not proceed hoping for the best.

That single rule converts an app redesign from a fleet-wide disaster into a notification. Without it, a moved button means four hundred businesses quietly get the wrong field changed and nobody finds out for a week.

Two: verify on a different path than you wrote

After the phone changes the hours in Toast, something else — a browser, on the public page — goes and reads them back. Different route, different software, independently.

If the verifier is the same thing that did the writing, it is self-certification with extra steps.

Writing through the phone and confirming through the browser is what makes the result evidence instead of a claim.

The owner does not get told "done." They get told "**Toast now shows Monday to Friday, nine to five**" with a screenshot attached and a reference number. And the whole chain — who asked, what authorised it, what ran, what came back — is written to a record that is added to and never edited.

Every competitor will happily tap a button and report success. None of them refuse to act because the screen changed, and none read the result back on a different path.

THE ONE THING WITH NO
UPSTREAM

The way in: email

Driving the phone is how things get *changed*. Getting information out of a business is a separate problem, and it has an answer nobody can revoke.

Every vendor a business uses already emails them. The booking confirmation. The daily sales summary. The payout notice. The cancellation. The review notification. That inbox is a complete picture of the business's operations, delivered automatically, every day, for free.

Those emails get forwarded to a parser. That is the whole integration, and it cannot be turned off — not without the vendor breaking their own product.

Getting it set up, ranked by friction

- 1 **Add a second notification address** inside the vendor's own settings. Toast, Square and FareHarbor all allow it. Two clicks, no Gmail involved, nothing to install.

- 2 **A forwarding rule** in the email they already use. Two minutes in a screen they already know.

- 3 **A script** — last resort only. "Run this script" kills the sale at the exact moment the product is supposed to feel like nothing.

The rule that cannot be added later

Every value is stored as an **observation**, with its source and its timestamp, and nothing is ever overwritten. The canonical answer is computed from the observations, not written over the top of them.

If values get written directly for six months and someone then decides they want to know where each one came from, there are six months of data with no origin and no way to reconstruct it.

Provenance is the one thing that must be built before the first email arrives.

What the data becomes

Once the emails are parsed, three things become possible that were not possible before, for anyone, at any price.

Real-time availability

A rental operator declares six jet skis. FareHarbor emails that two booked from ten to twelve. Booking.com emails that one booked from ten-thirty. A cancellation releases one. The answer is four, live, on their own website.

The thing no platform can do: **FareHarbor does not know about the Booking.com reservation.** No single channel sees all channels. The business does, and now so does their own site — which is the only way their site can ever compete with an agency taking twenty percent.

Charter doing \$400,000 a year at 20% commission	\$80,000
Moving 30% of bookings direct	\$24,000 back
What the subscription costs for a year	\$3,588
Return on one line item	6.7 : 1

Item-level verified reviews

The QR code on the table is scanned at a time, by a table, whose check closes on a ticket. That means the system knows what was actually ordered — the shrimp scampi, at 7:42, table nine — and a review can be attached to a dish rather than to a building.

A 4.3 tells a visitor nothing. "The bushwhacker is a 5.0 across two hundred and ten people who actually bought one" tells them where to go tonight. And in an internet filling with generated reviews, **a receipt is the last thing that cannot be faked.**

Canonical business truth

Hours that were changed and verified. Menus that are current because they were updated this morning. Prices that match the register. Published in a form any assistant can read directly. Ask any AI today who has crab legs nearby, or who has a six-top free at seven-thirty. Nothing on earth can answer, because the data has never existed in a form anyone could publish. That is the gap this fills, and filling it is worth more over ten years than the subscription is.

Four surfaces, one truth

There is one record of what is true about a business, and four ways to reach it. They are not four products — they are four doors into the same hallway, and every one of them produces the same structured request underneath.

- **Text.** Their own business number, on the SIM in the phone. Questions in, answers out, approvals by reply. This one works with no hardware at all, which is why it ships first.

- **Voice.** Wake word, listen, understand, speak. Most commands in a business are eight fixed sentences, so most never need a model at all.

- **The screen.** The wall kiosk. Not a dashboard — a short list of things that need an OK, approved with coffee in hand.

- **Another agent.** The machine-readable door. Somebody else's assistant asks a question and gets an answer without ever touching the data behind it.

And two halves that must never be confused

The **private half** lives on the node: the books, the customer list, the credentials, the observations. It never leaves the building.

The **public half** is a signed, read-only projection: hours, menu, availability, verified ratings. It is published everywhere and given away for nothing.

The edge that serves the public half has **no code path back into the node**. Not a permission setting — there is no route, because none was built. That is what makes "we cannot see your data" a structural fact instead of a promise in a privacy policy.

The apps on top

Once there is a live record of a business, applications stop being integrations and start being views. Each one is small, because the hard part already happened.

- **QR menu.** Always current, because it is the same menu that just got pushed to Google. Also the thing that closes the loop between what a guest looked at and what they ate.

- **Real-time availability.** On the business's own site, across every channel at once.

- **Verified reviews.** Item-level, receipt-backed, published everywhere and owned by the business.

- **Direct booking.** Zero percent. Keep the existing platform; this is the shop window, not a replacement.

- **Loyalty and points.** A co-op program that works across businesses in a town — which matters in a place people visit once a year and would otherwise never earn anything.

- **Referrals and attribution.** Authentic reviews turn into authentic referrals, and the transaction can be traced to the person who sent it.

- **The song request board.** QR code on the stage, a request, a payment straight to the artist, the confirmation email tying it back. The artist becomes a distribution channel.

- **Text marketing.** From the real number, to a list the business owns outright.

Every one of these makes the next one cheaper, because they share the same record, the same identity, the same permissions and the same proof.

The platform and the store

A business that runs three of those apps has effectively moved its operations onto the box. At that point the box stops being a tool and becomes a place other people can build.

One login, many apps

Everything arrives from one catalogue, the way apps arrive on a phone. Search, install, done — no repository, no command line, no version to choose. What gets listed has been scanned, signed, and tested against that exact machine first.

The permission model is the differentiator

The published packaging standards for agent plugins are good and worth adopting whole — they are backed by a steering committee that includes Amazon, Microsoft, OpenAI, Cursor and Vercel. But the specification says in its own words that it **contains no capability or permission system**. Installing is a single trust flag: yes or no.

That is fine on a developer's laptop. It is not fine on a machine holding a restaurant's books. So the capability list, the risk tiers, the sandbox, and the record of what every app actually did are the layer added on top — and that layer is the most defensible thing in the store.

Developers get paid and get nothing

An outside developer builds against capability names, not data. Their app asks a question; the answer crosses; the underlying records never do. **Agent to agent, with the business's agent as the intermediary that never hands anything over.**

That is the inversion. Every platform in the last twenty years paid developers in access to user data. This one pays them in money and gives them none.

WHY THIS COMPOUNDS

Every app built on the platform makes the platform more valuable to the next business, and every business added makes it more valuable to the next developer. Anyone who wants to sell an app to these businesses **has to sell them the platform first.**

How the money works

Two lines, and the second is the one nobody sees coming.

The subscription

A flat monthly fee per business. It pays the bills, it funds the churn absorption, and it scales with the number of customers rather than with their revenue. Hardware sells at a modest margin and is the acquisition channel, not the profit.

The rail

The platform charges **zero percent**. Developers pay nothing. Businesses keep their existing booking systems and pay no commission on anything.

The money is in the payments that flow through the ecosystem. A standard processing rate is 2.9% plus thirty cents. Volume gets that renegotiated. The difference between the negotiated rate and the posted rate is the margin, and it is invisible to everyone — the business pays what they always paid.

One percent of a very large number is a large number, and high-transaction verticals — charters, hotels, rentals, restaurants — are exactly where the volume is.

Platform fee charged to businesses	0%
Platform fee charged to developers	0%
Spread on processed volume	~1%
Budget for disputes and reserves	0.1–0.3%

That last line is the one that turns a full point into roughly seven tenths of one. With destination charges the platform carries dispute liability by default, plus negative balances when a business closes holding deposits. It is survivable and it must be budgeted from the first dollar, not discovered in year two.

Charging nothing is not generosity. It is how you get the volume that makes the spread worth having — and it is a price no competitor funded by a percentage can match.

ZERO TO ONE

The box itself

The operating system is not a collection of packages that drift apart on every machine. It is **one signed image with one version number**, byte-identical everywhere.

Updates do not patch. The machine replaces itself with the next image, reboots, checks its own health, and if anything is wrong it puts itself back the way it was. The owner never learns a problem occurred.

That single architectural choice is what makes everything else affordable. A support call starts with a version number and is reproducible in ten minutes on identical hardware, instead of being an archaeology project on a machine that has never existed before and never will again.

What the owner sees

One application, full screen, from the moment it boots. No desktop behind it, no file manager, no terminal, nothing that looks like a computer. A short list of things that need approval, and a way to ask for anything else.

And because they own it, there is a deliberate, documented way to the full Linux underneath. Held down, typed in — never stumbled into by a dropped tray.

Four claims almost nobody else can make

- **There is no remote access.** Not a promise not to look. There is no door, because it was never built — which also means it cannot be compelled, stolen, or misused.
- **An update cannot break it.** Failed health check, automatic rollback, invisible.
- **Lapsed means frozen, not bricked.** It runs forever and stays patched for security. It simply stops getting new things.
- **Backups go where they choose.** Their own drive, their own storage, encrypted with a key that is never held on the other end.

The harness

The thing being sold is not a model. Models change every six months and whichever is best today will not be best next year. What is being sold is the harness — the thing every model plugs into.

On the box: a small model always resident that decides what handles each request, one strong model held in memory, and a library of specialisations that swap in and out. The swap is the trick — loading a different model costs seconds, but swapping a small adapter on top of a loaded one costs about two hundred milliseconds. That is the difference between a library and a queue. Alongside it: the memory that belongs to this business and no other, the permission layer, the record of everything every agent did, and a bridge to any cloud model the owner wants to use with their own key — never proxied, never skimmed, never logged on the way through.

THE STRATEGIC BET

Every other platform is racing to own the endpoint so their model owns the customer. A machine that lets you swap freely gets **more valuable as that race accelerates**, while every locked-in machine gets less. That is a bet that compounds in your favour with time.

And the agent is deliberately stupid

It reads a column of structured data. It is strict. It is the same for every business. It does not improvise, it does not reason its way into a new approach, and it does not get creative with a price. That is not a limitation — it is the reason the thing can be trusted with a business, and it is why the words "artificial intelligence" appear nowhere in the patent claims.

The name and the fight

Steve Jobs left Apple, took Unix, and built NeXT — the operating system Apple still runs on today. He covered up where he found it. This does the opposite: it stays visibly Linux, and the credit stays with the people who built it.

That is not sentiment, it is the strongest available position. Wall Street runs on Linux. So does every AI company currently telling small businesses to trust the cloud. Building on the same foundation and saying so out loud is credibility that cannot be bought, and the instinct to rebrand it as a proprietary OS would throw it away.

The company is **A NEXT GENT**. The box is **Ghost**. Version one is the **Ghost of Lisa** — the Apple machine that failed because it was too early, that contained most of what computers would become, and that got buried in a landfill.

The advertisement

Apple's 1984 commercial was aimed at IBM. This one is aimed at gatekeepers — at an industry where every company is racing to get inside a business so the business's data becomes theirs. At the end, the screen does not get smashed with a hammer. It just shuts off.

It's not my computer. It's theirs. That sentence is the entire brand, and it is the one thing no cloud company can say back.

THE POSITION

Who buys it, and how

Small businesses are the customer nobody in technology has ever seriously served, for a reason that used to be true: acquisition cost. You cannot afford a salesperson for a two-hundred-dollar-a-month customer, and that customer will not self-serve.

Which is exactly why a founder who has sold cars and sold door to door for twenty years is the thing this company has that a better-funded competitor cannot buy at any price.

Density beats spread

A chamber of commerce has three hundred to eight hundred members, wants something of value to offer them, and has no technology of its own. One chamber agreement is a city's worth of warm introductions from a trusted party.

The same shape works for tourism boards, marina associations, restaurant associations and downtown districts. Two hundred businesses in one town is worth far more than two hundred scattered, because density is what makes the directory work and what makes a developer believe there is a market.

The vertical is the unit of work

A rental manager in Destin, Gulf Shores and Panama City is the same schema, the same procedures, the same emails. Once a vertical is built, customer two through two hundred cost effectively nothing regardless of where they are.

And it can be resold

White label, the way other platforms let an agency operate under their own brand. Trained salespeople from rental operations. Anyone who already sells to these businesses can sell this to them, and the product is easier to demonstrate than anything else they carry.

What's owned, what's given away

Almost all of it is given away. That is not a concession — it is the same trade Red Hat made, and it works for the same reason.

Given away

The source. The architecture. The capability names. The projections. Distribution of the business's own data to every directory that will take it, for nothing. And a share of revenue back to the open source projects the whole thing is assembled from, once there is revenue to share.

Held

- **The brand.** Anyone can rebuild the software. Nobody else can be A NEXT GENT. That is the same moat Red Hat has and it is more durable than any code.

- **The recordings.** The accumulated map of how every app works, kept current across hundreds of businesses. It grows every day, it cannot be bought, and it is worth more the longer it runs.

- **The pipe.** The signing keys, the channel, the entitlement. Anyone can rebuild the image; nobody else can publish to the machines.

- **The method.** Provisional patents on the system and method for build-integrated remote control — written deliberately without artificial intelligence in the claims, which puts them outside a very crowded field of prior art and describes a concrete mechanism instead of an aspiration.

ON BEING COPIED

Someone will rebuild the image for free. That is the CentOS story and it is survivable, because the free rebuild does not come with the hardware, the recordings, the fix pipeline, the warranty, or the phone number. The defence was never legal. It is that the thing being sold is a service that has to be performed every week.

What exists today

An honest inventory, because a plan that overstates its starting position is worthless.

Provisional patents filed	yes
Written architecture and component ledger	complete
External open-source components identified	281
Of those, ever actually running	~80
Needed for the first working demonstration	2
Existing repositories under the company	114
Flow engine, signing and packaging code	written, untested on hardware
Businesses running it	none yet
Relationships in the target market	real

The architecture is further along than the implementation, and that is the correct thing to be honest about. The documents have reached the point where they are auditing each other. None of them has a customer in it.

The order to build it

Ruthlessly sequenced. Each step produces something that works before the next one starts, and the early steps need no hardware and no operating system.

- 1 **Prove a phone can still be driven.** Real apps, real accounts, an accessibility service. One afternoon. Everything below depends on it.

- 2 **Change one business's hours from a command line, and read it back in a browser.** That is the whole product with nothing around it.

- 3 **Put the gate in front.** Who is asking, may they, how risky, and does a person have to say yes first.

- 4 **Put the record behind.** Append-only, with the screenshot and the verdict.

- 5 **Add the text door.** Their SIM, their number, approvals by reply. Now it is sellable with no hardware at all.

- 6 **Turn on the email pipe.** Observations, provenance, canonical values. This is the step that turns an automation tool into the data layer.

- 7 **Build the fix pipeline.** Nightly checks, one re-recording, signed, pushed to everyone. **Now the subscription is real.**

- 8 **Ship the appliance.** One image, health checks, rollback. The company, rather than the product.

- 9 **Entitlement and channels.** Who is paid up, what may this machine install, staged rollout.

- 10 **Open the store.** The manifest, the permission model, the certification gate, the first outside developer.

Steps one through five are the product. Six and seven are the business. Eight through ten are the company. Most of the value is in step one, and it has still not been run.

The one test that decides everything

There is a single unverified assumption underneath this entire document, and it can be settled in an afternoon for the price of a phone.

Install the applications a Destin charter operator or a 30A restaurant actually uses — the real ones, not similar ones. Turn on an accessibility service. Watch what still runs.

Some categories of application detect an active accessibility service and refuse to operate.

Banking is the strictest. Payment and point-of-sale software is the open question, and it is precisely the software that matters most here.

WHY THIS IS FIRST AND NOT FIFTH

If those applications block it, the mechanism has to change — and it is far better to know that in week one, with a forty-dollar phone, than in month nine with hardware ordered and customers signed. **Nothing else in this plan is worth starting before this is answered.**

The other risks, ranked honestly

- **Terms of service.** Counsel has been engaged. The posture — their device, their account, their credentials, human speed, human approval — is deliberate and it is documented.
- **Support cost per customer.** The thing that kills small-business hardware companies. Every simplification is margin.
- **Working capital on hardware.** The factory is paid ninety days before the customer pays. Ship software first for exactly this reason.
- **Scope.** The directory, the points, the referrals, the smart home, the other verticals — none of them are wrong, and every one of them costs a month that does not exist yet.

If it works

The modest version: a hundred businesses, six hundred thousand to a million a year, a real company that pays well and is owned outright. Most founders would take that and most never get it.

The version worth building for is different, and it is not about the box at all.

If a few thousand small businesses publish live, verified, item-level truth about themselves because doing so saves them money, then the answer to *where should I eat tonight* and *who has a boat free at two stops* being a guess assembled from stale listings. It becomes a fact, sourced from the business, consented to by the business, and owned by the business.

Whoever holds that becomes infrastructure the assistants depend on rather than a directory competing for attention. And because the data lives on the businesses' own machines and is published rather than hoarded, holding it does not require becoming the thing this whole project was built against.

That was the idea in 2013. The hard part was never the directory. It was always the data — and this is finally a mechanism that produces it.

FOURTEEN YEARS

One last thing.

Every number in this document is arithmetic off a stated assumption, not a case study. The \$400,000 charter is an example with a real commission rate applied to it; the working price is \$299 a month. **Change the assumption and the conclusion changes with it** — that is the point of writing them down.

And the honest summary of where this stands: **the thinking is further along than the building**. Step one of nineteen costs an afternoon and a forty-dollar phone, and until it is run, everything above it is a well-argued hypothesis.